

Note: Same for flood fill multi jump we use visited array.

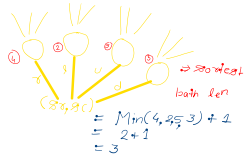
→ Same as flood fill just there are obstacle

```
public ArrayList<String> robotMaze(int[][] maze) {
    // base case
    if(maze == null || maze.length == 0 || maze[0].length == 0)
        return null;
    int n = maze.length;
    int m = maze[0].length;
    // if we reached the destination then return the path
    ArrayList<String> ans = new ArrayList();
    if(maze[0][0] == 1 || maze[n-1][m-1] == 1)
        return ans;

    public int findMaze(int sr, int sc, int dr, int dc, ArrayList<String> ans, String path,
        boolean[][] vis) {
        // if we reached the destination then
        // return the path
        if(sr == dr && sc == dc)
            return 1;
        // if we reached the destination then
        // return the path
        int count = 0;

        // if we reached the destination then
        // return the path
        for(int i = 0; i < 4; i++) {
            int nr = sr + di[i];
            int nc = sc + dj[i];
            if(nr < 0 || nr > n-1 || nc < 0 || nc > m-1 || vis[nr][nc])
                continue;
            if(maze[nr][nc] == 1)
                continue;
            vis[nr][nc] = true;
            path += di[i] + dj[i] + " ";
            count += findMaze(nr, nc, dr, dc, ans, path, vis);
            vis[nr][nc] = false;
            path = path.substring(0, path.length() - 1);
        }
        return count;
    }
}
```

Shortest path



lec 3 New topic x Recursion Trees

(n) Coin Change Permutation :-

2	3	5	7
---	---	---	---

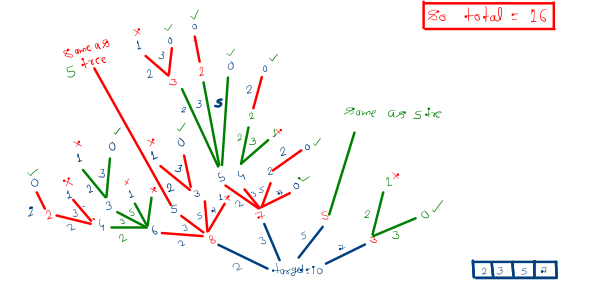
 Target = 10

Combination:

2	2	2	2
2	2	3	3
2	3	5	
3	7		
5	5		

5!/5! = 1
4!/2!*2! = 6
3! = 6
2! = 2
1! = 1
ans = 16

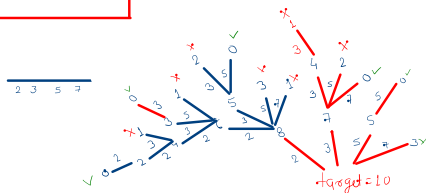
So total = 16



Code

```
public int coinChange(int[] coins, int amount) {
    // base case
    if(amount == 0)
        return 0;
    if(amount < 0)
        return -1;
    // if we reached the destination then
    // return the path
    int count = 0;
    for(int i = 0; i < coins.length; i++) {
        int nr = amount - coins[i];
        if(nr < 0)
            continue;
        count += coinChange(coins, nr);
    }
    return count;
}
```

Coin Change Combination Inf



→ In Combination we move only forward means element order is maintain for 2 is before 5 the each combination it is before 5

→ In the above problem as the coin is infinite so we can repeat the same coin while maintaining the order

```
public int coinChange(int[] coins, int amount) {
    // base case
    if(amount == 0)
        return 0;
    if(amount < 0)
        return -1;
    // if we reached the destination then
    // return the path
    int count = 0;
    for(int i = 0; i < coins.length; i++) {
        int nr = amount - coins[i];
        if(nr < 0)
            continue;
        count += coinChange(coins, nr);
    }
    return count;
}
```

→ in the code we used a for variable to track and maintain the order

Note: always choose Combination over permutation

Coin change Combination with Single coin

→ as we can only use the coins only one time so we pass the idx+1 is above code while calling it.



LeetCode 3

→ Code

322. Coin Change

Medium Types Compare

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return the **fewest number of coins** that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return `-1`.

You may assume that you have an infinite number of each kind of coin.

Example 1:

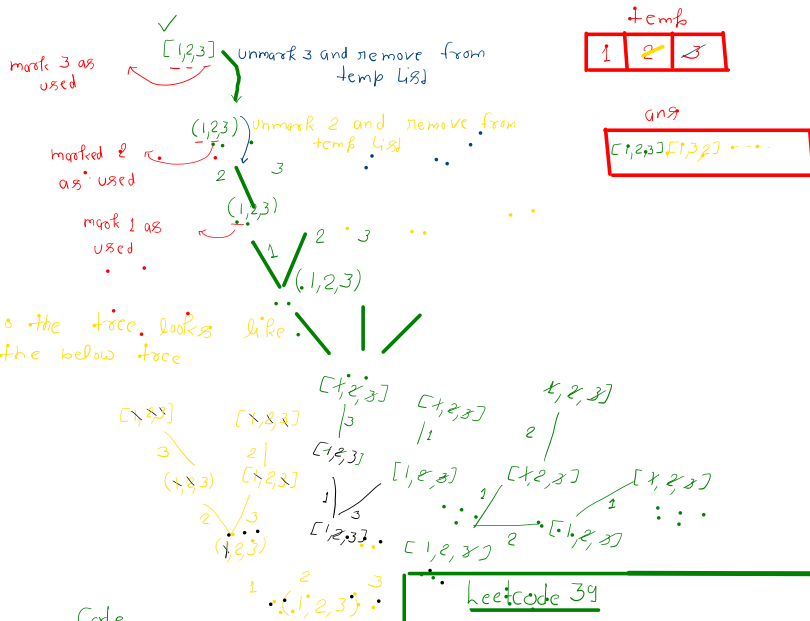
Input: `coins = [1,2,5], amount = 11`

Output:

Explanation: `11 = 5 + 5 + 1`

LeetCode 46

- our faith is that we pick the element from array and they will give you its all permutation
- when we picked the array element mark it so that we can not use it again while moving up to the stack. also add in arraylist.
- as we hit the base case copy the whole content of arraylist in our ans arraylist.
- as we going to the down of stack we unmark array element and remove from temp arraylist



Code

LeetCode 39

39. Combination Sum

Medium Types Compare

Given an array of **distinct** integers `candidates` and a target integer `target`, return a list of all **unique combinations** of `candidates` where the chosen numbers sum to `target`. You may return the combinations in **any order**.

The **same number** may be chosen from `candidates` an **unlimited number of times**. Two combinations are unique if the **indices** of at least one of the chosen numbers is different. The test cases are generated such that the number of unique combinations that sum up to `target` is less than 150 combinations for the given input.

Example 1:

Input: `candidates = [2,3,6,7], target = 7`

Output: `[[2,2,3],[7]]`

Explanation:

2 and 3 are candidates, and `2 + 2 + 3 = 7`. Note that 2 can be used multiple times.

7 is a candidate, and `7 = 7`.

These are the only two combinations.

```
public int permutation(int[] nums, int
idx, List<Integer> temp, List<List<Integer>> ans) {
    if (temp.size() == nums.length) {
        List<Integer> ans = new ArrayList<>(ans);
        ans.add(temp);
        return 1;
    }
    for (int i = 0; i < nums.length; i++) {
        if (temp.contains(nums[i])) continue;
        temp.add(nums[i]); //add the value in the stack
        permutation(nums, i+1, temp, ans);
        temp.remove(temp.size()-1); //remove the value
    }
    return 1;
}

public static int coinChangeCombination(int[] coins, int tar, String psf) {
    if (tar == 0) {
        System.out.println(psf);
        return 1;
    }
    if (tar < 0) return 0;
    int count = 0;
    for (int i = 0; i < coins.length; i++) {
        count += coinChangeCombination(coins, tar - coins[i], psf + coins[i] + " ");
    }
    return count;
}

public static int coinChangeCombination(int[] coins, int tar, String psf) {
    if (tar == 0) {
        System.out.println(psf);
        return 1;
    }
    if (tar < 0) return 0;
    int count = 0;
    for (int i = 0; i < coins.length; i++) {
        count += coinChangeCombination(coins, tar - coins[i], psf + coins[i] + " ");
    }
    return count;
}
```